Angular Complete Interview Question Bank

1. Angular Fundamentals, Architecture & CLI

- **What is Angular, and why is it used?**
  - Angular is a TypeScript-based open-source web framework used to build dynamic single-page applications (SPAs).
  - Angular is a component-based framework for building structured, scalable and single page web applications for client side.
  - In Single Page Application, we only have one page and the body content of that page will be updated as per the request.
- **What are the main features of Angular?**
  - Component-based architecture, Dependency Injection, Directives, Two-way data binding, Routing, and Forms handling.
- **What are Angular advantages?**
  - It is relatively simple to build Single Page Applications.
  - Flexible and structured client applications.
  - Cross platform and open source.
  - Writing reusable code is easy.
  - Testing is easy.
- **What is the difference between AngularJS and Angular? | What are the advantages of Angular over AngularJS?**
  - Angular JS supports JavaScript only, has MVC architecture, lacks CLI tool, lacks Dependency Injection, doesn't support mobile browsers, and is not so fast.
  - Angular supports JavaScript and TypeScript, uses component-based architecture, has CLI tool, uses Dependency Injection, supports mobile browsers, and is very fast.
- **Describe the Angular application architecture. | How would you design a reusable Angular application architecture? | Explain an Angular project architecture you worked on. | How do you structure a large front-end codebase (monorepo vs multirepo, feature-based folders, shared packages)?**
-
- **What is Angular CLI? | What is Angular CLI, and what can it be used for?**
  - The Angular CLI is a command-line tool for generating components, services, modules, and running Angular applications.
  - The Angular CLI (command-line interface) is a tool that you use to initialize and develop Angular applications directly from Terminal.
- **What are the building blocks of Angular?**
- **What is NPM?**
  - NPM is an online repository, from where you can get thousands of free libraries which can be used in your angular or node js projects.
- **Where to store static files in Angular project?**
  - Store static files in assets folder of the Angular project.
- **What is the role of Angular.json file in Angular?**
  - The angular.json file is the primary configuration file for an Angular project.
  - It contains all the links of all the files in Angular project.
- **How an Angular App gets Loaded and Started? What are index.html, app-root, selector and main.ts?**
  - Request hits index.html page.
  - Index.html invokes main.js file (which is the javascript version of main.ts file).
  - main.ts compiles the web-app and bootstraps the app.module.ts file.
  - App-Module file will then bootstrap the app-component.
  - App-component HTML will be displayed.
- **What is a Bootstrapped Module & Bootstrapped Component?**

- The module which loaded first, when the Angular application starts is called Bootstrapped module (by default AppModule).
- The component which loaded first, after the module is called Bootstrapped component (by default AppComponent).

2. Components & Modules

- **What are Angular components?**
  - Components are UI building blocks in Angular, defined by a TypeScript class, HTML template, and CSS styles.
  - Components are the most basic UI building block of an Angular app.
  - Combination of all these 4 files (css, html, spec.ts and ts) is one app-component.
- **What is a module in Angular? | What is app.module.ts file?**
  - A module is a container that organizes an application into cohesive blocks. It uses @NgModule to define dependencies.
  - Module is a place where you can group the components, directives, pipes, and services, which are related to the application.
- **What is a Selector and Template?**
  - A selector is used to identify each component uniquely into the component tree.
  - A Template is a HTML view of an Angular component.
- **What are standalone components in Angular?**
- **How do you dynamically create and load components in Angular?**
- **How do you design reusable components so they remain flexible without becoming "god components"?**
- **Why Angular is moving away from NgModules?**

3. Data Binding & Component Communication

- **Explain data binding in Angular.Explain different types. | 10. What are the 4 types of Data Binding?**
  - It synchronizes data between the model and view. It includes: Interpolation, Property Binding, Event Binding, and Two-way Binding.
  - Data binding is the way to communicate between your typescript code of your component and html view of component.
- **What is interpolation in Angular? | What is String Interpolation in Angular?**
  - String Interpolation is a one-way data-binding technique that is used to transfer the data from a TypeScript code (component) to an HTML template (view).
  - String interpolation can work on string type only and is represented inside double curly braces {{data}}.
- **Difference between Property Binding and Interpolation? | What is Property Binding in Angular?**
  - Property binding is a superset of interpolation that can set an element property to a non-string data value like Boolean.
  - Use interpolation for strings and property binding for other types.
- **What is Event Binding in Angular?**
  - Event binding is used to handle the events raised by the user actions like button click.
- **What is two-way binding, and how do you implement it in Angular?**
  - Two-way data binding in Angular will help users to exchange data from the view to component and then from component to the view at the same time.
- **How do you pass data between parent and child components? | How do you pass data from Parent to Child? | How do you send data from Child to Parent? | How do you share data between unrelated components?**
  - Parent component is the outer component and child components are the components inside the parent.
  - Ways to communicate include: @Input, @Output, Ng-content, @ViewChild, @ViewChildren, @ContentChild, and @ContentChildren.

- **What is @Input()? | What is @Input Decorator? How to transfer data from Parent component to Child component?**
  - @Input decorator is used to pass data from parent component to child component.
- **What is an EventEmitter in Angular? | What is @Output()? | What is @Output Decorator and Event Emitter?**
  - @Output decorator and event emitter together are used to pass data from child component to parent component.
- **What is Content Projection in Angular? | What is <ng-content>?**
  - ng-content is used to insert the dynamic/variable html content from one component to another component.
- **What is Template Reference Variable in Angular?**
  - Template Reference Variable in angular is used to access all the properties of any element inside DOM.
- **What is the role of ViewChild in Angular? | How to access the child component from parent component with ViewChild?**
  - @ViewChild decorator is used to access elements of html template file from the component typescript file.
  - To access the child component, write code inside the ngAfterViewInit lifecycle hook of the parent component.
- **What is the difference between ViewChild and ViewChildren? What is QueryList?**
  - @ViewChild can access only one element, whereas @ViewChildren can access a list of elements.
  - QueryList is used to manage the list of elements referenced by @ViewChildren.
- **What is ContentChild? | What is the difference between ContentChild & ContentChildren?**
  - ContentChild is used to send variable or projected content from one component html to another component class file.
  - @ContentChildren does the same thing but accesses a list of elements.
- **Compare ng-Content, ViewChild, ViewChildren, ContentChild & ContentChildren?**

4. Directives & Pipes

- **What are directives in Angular? Name a few commonly used ones. are the type of directives?**
  - Directives are classes that modify the DOM. Types: Structural (*ngIf, *ngFor) and Attribute (ngClass, ngStyle).
  - Directives are classes that add additional behavior to elements in your Angular applications.
  - Types include Structural (change appearance of DOM by adding/removing elements), Attribute (change appearance or behavior), and Component (directives with its own template).
- **What is the difference between components and directives? | What is the difference between structural and attribute directives? |What is the difference between Component, Attribute and Structural Directives?**
  - Component directive is responsible for showing the first whole view; Structural adds/deletes html elements (starts with * sign); Attribute changes appearance/styles (set inside square brackets).
- **Explain: *ngIf, *ngFor, ngClass, ngStyle | Q23. What is *ngSwitch Structural directive? | What is [ngStyle] Attribute directive? | Q25. What is [ngClass] Attribute directive?**
  - *ngIf: adds or removes items based on if condition.
  - *ngFor: iterates a list of items to display them.
  - *ngSwitch: used in combination with *ngSwitchCase and *ngSwitchDefault.
  - [ngStyle]: updates styles of the HTML element.
  - [ngClass]: adds and removes CSS classes on an HTML element.
- **How do you apply conditional styling in Angular components?**
- **What are pipes in Angular? What are the types of Pipes & Parameterized Pipes?**
  - Pipes are simple functions which accept an input value and return a transformed value.
  - Types: Built-in (provided by Angular framework like lowercase) and Custom (developed yourself). Parameterized pipes are when parameters are passed to pipes.
- **How do you create a custom pipe in Angular? Difference between Pure and Impure Pipes?**

- **What is the purpose of the async pipe?**
- **What is Chaining Pipes?**
  - The chaining pipes use multiple pipes on a data input.
- **What are Control Flow statements? (@if, @for, @switch)**

5. Lifecycle Hooks

- **What are lifecycle hooks in Angular?**
  - A component goes through several stages from creation to destruction called lifecycle hooks (Component instantiating, Rendering HTML, Creating child components, Destroying the component).
- **Explain ngOnInit and ngOnDestroy lifecycle |.What is ngOnInit life cycle hook in Angular?**
  - ngOnInit signals activation of the created component, is called after ngOnChanges, and is called only once.
- **What is the purpose of ngAfterViewInit?**
- **What is ngOnChanges life cycle hook in Angular?**
  - The ngOnChanges is the first life cycle hook, which angular fires when it detects any changes to the input property.
- **Difference between Constructor and ngOnInit()?**
  - Constructor is a TypeScript class method that automatically gets called upon instantiation (before any lifecycle hook) and is used to inject dependencies.
  - ngOnInit is a lifecycle hook signaling component activation (called after ngOnChanges). It is used for business logic since the component is fully ready.

6. Decorators

- **What are Decorators (@Component, @Injectable)? | What is Decorator?**
  - Angular decorators store metadata about a class, method, or property.
  - Metadata is "data that provides information about other data".
- **What are the types of Decorator?**
  - Class Decorators (@NgModule, @Component, @Injectable, @Directive, @Pipe).
  - Property Decorators (@Input, @Output, @ContentChild, @ViewChild).
  - Method Decorators (@HostListener).
  - Parameter Decorators (@Inject, @Self, @Host).

7. Dependency Injection & Services

- **What is the purpose of services in Angular? | What are Angular services, and why are they useful? Explain Services with Example?**
  - Services allow shared logic and data management across components using Dependency Injection.
  - A service is a typescript class and reusable code which can be used in multiple components.
- **What is Dependency Injection in Angular? | How to use Dependency Injector with Services in Angular?**
  - A design pattern in which dependencies are injected into a class instead of being instantiated within the class.
  - 4 Steps: 1. Create the service. 2. Set providers inside the component decorator. 3. Assign Service type as a property in the constructor parameter. 4. Call the service method.
- **How to create Service in Angular?**
  - Use the command: ng generate service [name].
- **What is a Singleton Service in Angular? | What is a Singleton pattern, and how does Angular use it?**
  - A service that is instantiated only once and shared across the entire application.
- **How does Angular handle Dependency Injection hierarchy? | What is Hierarchical Dependency Injection?**
  - If you inject the service in parent component, it will be available to all its child components. If injected directly in child, it will not be available for parent/siblings.

- **What is Provider in Angular?**
  - A provider is an object declared inside decorators which inform Angular that a particular service is available for injecting.
- **What are multi-providers in Angular?**
- **What is the role of @Injectable Decorator in a Service?**
  - With @Injectable decorator one service can be used by another service.
- **What is forwardRef in Angular?**

8. Routing & Navigation

- **How does Angular handle routing? | What is Angular Routing? How to setup Routing?**
  - Routing helps in navigating from one view to another view with the help of URL in a single page.
- **What is router outlet?**
  - Router-outlet acts as a placeholder to load different components dynamically based on the activated component via Routing.
- **What are router links?**
  - RouterLink is used for navigating to a different route.
- **What is a Router Guard in Angular? Explain interceptors/guards/resolvers—where have you used each and why? | What is Auth Guard?**
  - Auth guard is used for guarding the routes (e.g., redirecting users to a login page if not authenticated).
- **Difference between: CanActivate, CanDeactivate, Resolve**
- **What is an Angular Resolver?**
- **What are query parameters in Angular routing?**
- **What is ActivatedRoute in Angular?**
- **What is lazy loading in Angular?** |
- **Advantages of Lazy Loading?**
- **What is Preloading Strategy?**
- **How do you handle routing, code splitting, and lazy loading in React apps?**

9. RxJS, Observables & Signals

- **What is RxJS in Angular? | What is an Observable? | What is RxJS? | What is Observable? How to implement Observable?**
  - RxJS is a javascript library that allows working with asynchronous data stream via observables.
  - Observables stream data to multiple components continuously (e.g., receiving data from APIs).
  - Implementation Steps: 1. Import Observable from RxJS Library. 2. Create Observable & Emit Data. 3. Subscribe the Data.
- **What are Observables and Promises? | Difference between Observable and Promise? | What is the difference between Promise and Observable?**
  - Observables emit multiple values over time (stream), are lazy (don't execute until subscribed), and can be cancelled.
  - Promises emit a single value, execute immediately, and are not cancellable.
- **What are Asynchronous operations?**
  - In asynchronous operations, tasks are executed parallelly taking less total time compared to synchronous sequential operations.
- **What is Subject? | Difference between: Subject, BehaviorSubject, ReplaySubject, AsyncSubject Subject vs BehaviorSubject vs ReplaySubject**
- **Explain operators: map, filter, switchMap, mergeMap, concatMap, exhaustMap | 3. RxJS Operators (switchMap, mergeMap) | How have you used RxJS in production? Explain higher-order mapping operators and cancellation (switchMap vs mergeMap vs concatMap).**
- **When would you use switchMap? | Difference between switchMap and mergeMap?**
- **A component makes multiple API calls. Which RxJS operator would you use and why?**
- **What is forkJoin()? | What is combineLatest()? | What is debounceTime()?**

- **What is the purpose of catchError in RxJS?**
- **What are Signals? | Signals vs RxJS | Difference between Signals and RxJS?**
- **What is computed()? | What is effect()? | When should Signals be used?**

10. Forms

- **What is a template-driven form in Angular?** | **What is a reactive form in Angular?** | **What are Angular Forms? What are the type of Angular Forms?**
  - Angular forms handle user input (login, registration).
  - Template driven forms have logic in HTML templates.
  - Reactive forms have logic written in the component typescript class file.
- **What is the difference between template-driven and reactive forms?** | **37. Difference between Template-Driven and Reactive Forms?** | **What's your approach to forms (template-driven vs reactive), dynamic forms, and validation?** | **What is the difference between Template Driven Forms & Reactive Forms?**
  - Template-driven uses FormsModule and is meant for simple applications.
  - Reactive uses ReactiveFormsModule and is preferred for complex apps with more controls.
- **Why Reactive Forms preferred in enterprise apps?** | **Reactive Forms**
- **How to setup Template Driven Forms?** | **How to setup Reactive Forms?**
- **What is FormGroup in Angular Forms?** | **What is FormGroup?** | **What is FormControl in Angular?** | **What is FormControl?** | **What is Form Group and Form Control in Angular?**
  - FormGroup is a collection of form controls. FormControls track individual elements. Both inherit from AbstractControl to track states/values.
- **What is the difference between ngModel and formControl?**
- **What is FormBuilder in Angular?**
- **How do you validate forms in Angular?** |**How to apply Required field validation in template driven forms?** | **How to do validations in reactive forms?**
  - Apply required keyword to invalidate the form until filled in templates, or programmatically in Reactive.
- **How do you create custom validators?** | **Async Validators?**

11. HTTPClient & API Integration

- **What is HttpClient in Angular?** | **What is the role of HttpClient in Angular?**
  - HttpClient is a built-in service class used to perform HTTP requests (GET/POST/PUT/DELETE) to send/receive data from APIs.
- **How do you make an API call using HttpClient?** | **What are the steps for fetching the data with HttpClient & Observable?**
  - Create a service and configure observable/HTTPClient -> use Subscribe method in component -> use for loop to display in HTML.
- **What are interceptors in Angular?**
  - Interceptor is a special Angular Service used to intercept all request and response calls and modify them (e.g., attach auth tokens globally).
- **How do you handle authentication in Angular?**
  - Authentication verifies identity; Authorization allows resource access based on role.
- **What is JWT Token Authentication in Angular?** | **How to implement the Authentication with JWT in Angular?** | **How do you add JWT Token in every request?** | **Which part of the request has the token stored when sending to API?**
  - JWT authenticates user credentials. Tokens are sent in the HEADER part of the request.
- **What are the parts of JWT Token?**
  - Header, Payload, and Signature.
- **How to Mock or Fake an API for JWT Authentication?**
- **What is Postman?**
  - Postman is a tool for testing APIs.

- **How do you handle error messages globally in Angular? | 9. What is your approach to handling errors in Angular services? | How do you handle API errors globally? | Q54. How to do HTTP Error Handling in Angular?**
    - HTTP Error Handling uses catchError and throwError functions from rxjs.
- **How to Retry automatically if there is an error response from API?**
    - Use the Retry operator from the RxJS library.
- **How do you handle file uploads in Angular?**

12. Performance, Change Detection & Optimization

- **What is Change Detection in Angular? | Change Detection & OnPush**
- **What is the difference between OnPush and Default Change Detection? | 55. Difference between Default and OnPush?**
- **When should OnPush be used?**
- **What does trackBy do?**
- **What is AOT (Ahead-of-Time) compilation? | What is the difference between JIT and AOT in Angular?**
    - JIT compiles the app in the browser at runtime; AOT compiles the app/libraries at build time. Loading in AOT is much quicker.
- **What is ViewEncapsulation in Angular?**
- **How do you improve performance in a large Angular application? | How to optimize large Angular applications? | How do you improve First Load Performance? | How would you debug a slow Angular application? | How do you manage performance in Angular (OnPush, trackBy, lazy loading, change detection pitfalls)?**
- **How does Angular handle memory leaks? | How do you debug memory leaks in Angular? | How do you handle memory leaks?**
- **How would you handle large datasets in an Angular application? | How does Virtual Scrolling improve performance? | How would you optimize a page with 10,000 records? | What are your strategies for large tables/graphs/dashboards (virtualization, memoization, pagination)?**
- **What is Tree Shaking in Angular? | How do you optimize bundles (tree shaking, code splitting, dependency audits)?**
- **How do you optimize Angular applications for performance and scalability? | How do you track and optimize performance metrics in Angular? | How do you measure front-end performance (Lighthouse, Web Vitals, profiler)? What metrics matter most for your apps?**

13. NgRx & State Management

- What is state management in Angular? | Explain your approach to state management at different scales (local state, context, Redux/NgRx, server state).
- Why use NgRx? | What problem does NgRx solve?
- What are Actions, Reducers, and Effects in NgRx? | Explain: Store, Action, Reducer, Effect, Selector
- What is a Store in NgRx?
- Difference between Service State Management and NgRx? | When would you choose NgRx over services?
- How do you manage state in Vue (Pinia/Vuex), and what patterns keep it maintainable?
- How do you manage state and side-effects in React (useState/useReducer, Context, Redux, React Query)?

14. Testing & Quality

- What testing tools are used in Angular?
- What is Jasmine and Karma?
- How do you test services in Angular? | Testing: how do you write unit tests for components/services? What do you mock and what do you integration-test?
- How to mock HTTP requests in unit tests?
- What are E2E tests in Angular? | What is your test pyramid for front-end applications (unit/integration/e2e)?

- How do you debug an Angular application effectively?
- How do you test API integration and error states (timeouts, 4xx/5xx, empty data)?
- How do you make unit tests stable and not brittle (selectors, fixtures, mocking)?
- Testing approach (Jest/RTL): what do you unit test vs integration test in React?
- How do you enforce code quality (linting, formatting, type checks) in CI? | How do you ensure quality during fast delivery (PR checks, review checklist, definition of done)?

15. Upgrades, SSR & Modern Angular

- What is ng update in Angular? | Describe an upgrade you led (e.g., Angular 9 → 12). What broke, and how did you de-risk it?
- What is Angular Universal?
- What is Server-Side Rendering (SSR)? | 75. What is Server Side Rendering (SSR)?
- How does Angular handle SSR with Angular Universal?
- What is prerendering in Angular Universal?
- Explain Deferred Loading (@defer)?
- What is Ivy in Angular?
- What is an Angular schematic?
- What is ng-packagr in Angular?
- How do you create a PWA (Progressive Web App) with Angular?
- What is the purpose of tsconfig.json in Angular?
- What is differential loading in Angular?
- What are Web Components, and how can Angular elements be used as Web Components?
- What is the Shadow DOM, and how does Angular use it?

16. TypeScript & JavaScript Fundamentals

- **What is Typescript? Or What is the difference between Typescript and Javascript?**
  - TypeScript is an open-source, strongly typed superset of JavaScript with OOPS features that detects errors at compile time.
- **How to install Typescript and check version?**
- **What is the difference between let and var keyword?**
  - var is global/function scoped and let is block scoped.
- **What is Type annotation?**
  - Helps compiler check variable types to avoid errors (e.g., assigning a string to a number variable throws an error).
- **What are Built in/ Primitive and User-Defined/ Non-primitive Types in Typescript?**
  - Built-in (primitive) stores simple data like strings, numbers, booleans.
  - User-Defined (non-primitive) stores complex data like arrays, enums, classes.
- **What is "any" type in Typescript?**
  - Bypasses typechecking from the compiler, allowing extreme flexibility (but removing safety).
- **What is Enum type in Typescript?**
  - Allows defining a set of named constants.
- **What is the difference between void and never types in Typescript?**
  - void means no type (returns empty). never means it will *never* return (used to throw errors).
- **What is Type Assertion in Typescript?**
  - Technique that informs the compiler about the type of a variable manually (<String>).
- **What are Arrow Functions in Typescript?**
  - A compact alternative syntax to traditional function expressions.
- **What is Object Oriented Programming in Typescript?**
  - Used to design structured apps (Classes, Objects, Encapsulation, Inheritance, Polymorphism, Abstraction).
- **What are Classes and Objects in Typescript?**
  - Classes are blueprints; Objects are instances of a class.

- **What are Access Modifiers in Typescript?**
    - Public (accessible everywhere), Private (same class only), Protected (same class + child/derived classes).
- **What is Encapsulation in Typescript?**
    - Wrapping data and functions to prevent direct access (data security).
- **What is Inheritance in Typescript?**
    - Feature where derived classes acquire properties/methods of a base class (code reusability).
- **What is Polymorphism in Typescript?**
    - Same name method can take multiple forms or different implementations.
- **What is Interface in Typescript?**
    - A contract where methods are only declared. Forces implementing classes to stay in sync.
- **What's the difference between extends and implements in TypeScript**
    - extends is used for base class inheritance; implements is used for interface inheritance.
- **Is Multiple Inheritance possible in Typescript?**
    - It is only possible via Interfaces, not base classes.
- **Explain event loop, microtasks vs macrotasks, and how Promises/async-await fit in.**
- What are closures and where have you used them in real code?
- Explain prototype inheritance vs class syntax; when do you prefer composition?
- TypeScript: how do you design types for API responses and avoid “any” creep?
- Explain generics with an example from your code (e.g., reusable table/form components).

17. Backend, Cloud, Frameworks & General UI Architecture (Scenario/Leadership)

- How do you prevent unauthorized access in Angular? | How do you implement role-based authorization?
- How would you integrate Angular with a backend (Node.js, .NET, etc.)?
- How would you implement internationalization (i18n) in Angular?
- What is the purpose of the Renderer2 service?
- How do you secure an Angular application? | Explain common web security risks (XSS, CSRF) and what you do in the front end to reduce risk.
- Walk me through your last 2–3 years of work. What were you personally accountable for?
- What kind of products have you built (B2B/B2C/internal tools)? Which had the highest scale or complexity?
- Which framework do you consider your strongest today (Angular/React/Vue) and why?
- Describe a decision you made that improved delivery speed or quality across the team.
- When do you choose micro-frontends? What integration patterns have you used (module federation, iframe, custom elements)?
- How do you handle configuration-driven UIs (feature flags, dynamic forms, tenant-based config)?
- What are your standards for accessibility and design systems? How do you enforce them?
- How do you version and publish shared UI libraries, and prevent breaking changes across apps?
- Describe how you design a React component API (props, composition, render props/hooks) for reuse.
- What are common causes of unnecessary re-renders in React? How do you detect and fix them?
- Explain hooks rules and a real bug you’ve seen due to dependencies in useEffect/useMemo/useCallback.
- If you migrated a feature from Angular/Vue to React, what were the key differences in patterns?
- Where does Vue work best for you compared to Angular/React?
- Nuxt: how do you decide between SSR, SSG, and SPA mode? What trade-offs did you face?
- How do you handle component communication in Vue (props/events, provide/inject) and avoid tight coupling?
- Vuetify/Tailwind: how do you balance speed of delivery with consistent UX and performance?
- How do you handle error boundaries and runtime errors in front-end apps?
- Describe a performance issue you diagnosed end-to-end (symptoms → root cause → fix).
- How do you design for observability (logging, tracing IDs, error reporting) in UI + backend integration?
- How do you design REST APIs to support UI needs (pagination, filtering, sorting, partial responses)?
- Tell me about a tricky integration issue between UI and backend and how you resolved it.
- How do you handle API versioning and backward compatibility for long-running enterprise apps?

- Node/Express: how do you structure middleware and error handling?
- Spring Boot: what patterns have you used for auth, controllers, and DTO mapping?
- Python services (Flask/FastAPI): how did you handle configuration, logging, and deployment readiness?
- Azure fundamentals: which services have you used directly (App Service, storage, CI/CD)? What was your role?
- Databricks: how did the UI interact with Databricks workloads (jobs, notebooks, APIs)?
- GCP services: which services did you integrate with, and what were the key constraints (IAM, networking, quotas)?
- How do you manage environment configs (dev/qa/prod) and secrets for front-end builds?
- You mention integrating AI-driven tools/agents to improve development velocity. What exactly did you implement (use cases, tooling, workflow)?
- How did you evaluate quality and risk (hallucinations)?

Top 30 Most Asked Angular Interview Questions (Experienced)

Angular Architecture

1. Explain Angular application architecture.
2. What are Standalone Components and their advantages?
3. How do you structure a large-scale Angular application?
4. Smart vs Dumb Components?
5. Container vs Presentational Components?
6. When would you use Angular Elements?
7. What is Dependency Injection and how does Angular implement it?
8. What is Hierarchical Dependency Injection?
9. Difference between `providedIn:'root'` and module providers?

Change Detection & Performance

10. How does Angular Change Detection work?
11. Difference between Default and OnPush strategy?
12. When would you use Signals over RxJS?
13. How would you optimize an Angular application with 10,000 records?
14. How do you improve First Load Performance?
15. How do you reduce bundle size?
16. What is lazy loading and how does it improve performance?
17. What causes memory leaks in Angular and how do you prevent them?

RxJS (Very Frequently Asked)

18. Difference between Observable, Promise, and Signals?
19. Explain `switchMap`, `mergeMap`, `concatMap`, and `exhaustMap`.
20. Which operator would you use for search/autocomplete and why?
21. How do you handle multiple API calls?
22. Difference between `Subject`, `BehaviorSubject`, and `ReplaySubject`.
23. How do you implement retry and error handling in RxJS?

State Management

24. When do you use NgRx?
25. NgRx vs Service with BehaviorSubject?
26. Explain Store, Actions, Reducers, Effects, and Selectors.
27. How would you manage state in a large enterprise application?

Routing & Security

28. Explain Angular Routing lifecycle.
29. Difference between CanActivate, CanDeactivate, Resolve, and CanMatch.
30. How do you secure Angular applications?
31. What is JWT authentication flow?
32. How do Interceptors help in security?

Forms

33. Reactive Forms vs Template-driven Forms?
34. How do you create custom validators?
35. How do you build dynamic forms?
36. How do you handle cross-field validation?

Component Communication

37. Parent to Child communication methods?
38. Child to Parent communication methods?
39. How do unrelated components communicate?
40. ViewChild vs ViewChildren?

Advanced Scenarios (Architect Round)

41. A dashboard is slow. How will you identify and fix performance issues?
42. Bundle size has become 8 MB. What will you do?
43. Users report memory increasing continuously. How will you debug?
44. You have to migrate Angular 12 to Angular 19+. What is your approach?
45. When would you choose Signals instead of NgRx?
46. How would you implement Micro Frontends in Angular?
47. How do you handle role-based access control (RBAC)?
48. How would you design a reusable component library?

Questions Asked in Almost Every Angular Interview If you have limited time, focus on these:

- ✅ Change Detection
- ✅ OnPush Strategy
- ✅ Signals
- ✅ RxJS Operators (`switchMap`, `mergeMap`)
- ✅ Subject vs BehaviorSubject
- ✅ NgRx Architecture
- ✅ Standalone Components
- ✅ Lazy Loading
- ✅ Interceptors
- ✅ Route Guards
- ✅ Reactive Forms
- ✅ Memory Leak Prevention
- ✅ Component Communication
- ✅ Dependency Injection
- ✅ Performance Optimization
- ✅ Large Dataset Handling
- ✅ Angular Project Architecture